Azure Databricks | DataFrame Practice

Cinema Booking Analytics • 25 practical exercises • CSV + nested JSON • No solutions

Dataset	Contents
cinema_bookings.csv	32 booking rows, including 2 repeated booking IDs; missing discount and payment values
cinema_master.json	6 cinema records; nested location object and facilities array

Getting started in Azure Databricks

Upload both files to a Unity Catalog volume, copy their actual paths, and run the following starter code in a Python notebook with compute attached. JSON is newline-delimited, so multiLine is not required.

```
csv_path = "/Volumes/<catalog>/<schema>/<volume>/cinema_bookings.csv"
json_path = "/Volumes/<catalog>/<schema>/<volume>/cinema_master.json"
bookings_df = spark.read.csv(csv_path, header=True, inferSchema=True)
cinemas_df = spark.read.json(json_path)
display(bookings_df); display(cinemas_df)
```

Submission: Use PySpark DataFrame operations. Submit the notebook with code and visible outputs. Do not modify the source files.

A. Read and inspect the data

1. Load cinema_bookings.csv into a Spark DataFrame with column names and inferred data types.
2. Load cinema_master.json into a second DataFrame. Display both DataFrames.
3. Print the schema of both DataFrames. Identify the nested field and array field in the JSON.
4. Display the first 10 booking records and count the total number of CSV rows.
5. Select only booking_id, movie_title, tickets and booking_status.

B. Filter, clean and transform

6. Filter bookings where booking_status is Confirmed.
7. Find bookings with more than 3 tickets and a ticket price above 200.
8. Count missing values in discount_pct and payment_method.
9. Create a cleaned DataFrame that replaces missing discounts with 0 and missing payment methods with Unknown.
10. Add gross_amount = tickets multiplied by price_per_ticket.
11. Add net_amount after applying discount_pct. Keep the amount as a numeric column.
12. Add a booking_size column: Large for 4 or more tickets, otherwise Regular.
13. Sort bookings by net_amount from highest to lowest and show the top 5.

C. Aggregations and joins

14. Count bookings for each booking_status.

15. For Confirmed bookings only, calculate total tickets and net revenue for each movie_title.
16. Find the top 3 cinema_id values by Confirmed net revenue.
17. Left join the bookings DataFrame with the cinema master on cinema_id.
18. From the joined DataFrame, show cinema_name, location.city, movie_title and booking_status.
19. Calculate Confirmed net revenue by city. Identify bookings with no matching cinema master record.

D. JSON, dates and DataFrame quality

20. Use the nested JSON field location.zone to display each cinema and its zone.
21. Explode the facilities array to produce one row per cinema and facility.
22. Convert booking_date and show_date into date columns and calculate days_between_booking_and_show.
23. Identify repeated booking_id values and remove duplicate bookings while retaining one row per booking_id.

E. Window functions and Delta

24. Use a window function to rank movies by Confirmed net revenue (highest first).
25. Save the cleaned, deduplicated, joined DataFrame as a Delta table and query five rows with SQL.

Final challenge (optional)

Build a clean cinema performance report: deduplicate bookings, fill missing values, calculate discounted revenue for Confirmed bookings, join cinema details, aggregate by city and movie, and save the result as a Delta table.

Note: The CSV intentionally contains an unmatched cinema ID (C999) to practice left joins and missing master data.